

Вложенные структуры данных

1. Что это такое? (Аналогия)

Представь обычный список как **коробку**, где лежат вещи. **Вложенный список** — это коробка, внутри которой лежат другие коробки поменьше.

Это нужно, когда данные имеют структуру. Например:

- **Список учеников в классе** — это просто список.
- **Список классов в школе** — это вложенный список (список списков учеников).

2. Вложенные Списки (Nested Lists)

Списки **изменяемы**, поэтому их чаще всего используют для создания таблиц, игровых полей или баз данных.

Как создать и получить доступ

```
# Игровое поле для "Крестиков-ноликов" (3x3)
matrix = [
    ["X", "O", "X"], # Индекс 0
    [" ", "X", " "], # Индекс 1
    ["O", " ", "O"]  # Индекс 2
]

# Чтобы достать элемент, нужно два индекса:
# [ряд][столбец]
print(matrix[0][1]) # Выведет "O" (нулевой ряд, первый элемент)

# Чтобы изменить элемент:
matrix[1][2] = "X"
```

3. Вложенные Кортежи (Nested Tuples)

Кортежи **неизменяемы**. Вложенные кортежи идеальны для данных, которые не должны меняться в процессе работы программы.

Пример: Координаты маршрута

```
# Маршрут: (широта, долгота)
route = (
    (55.75, 37.61), # Москва
```

```

    (59.93, 30.33), # Санкт-Петербург
    (54.31, 48.40) # Ульяновск
)

# Доступ такой же:
print(route[1][0]) # 59.93

```

На что обратить внимание: Если ты попытаешься написать `route[0][0] = 60.0`, Python выдаст ошибку. Это «замороженные» данные.

4. Как работать с ними через циклы

Для вложенных структур нам нужны **вложенные циклы**. Внешний цикл проходит по «рядам», а внутренний — по элементам внутри ряда.

```

office_plan = [
    ["Анна", "Иван"],
    ["Петр", "Мария", "Олег"],
    ["Елена"]
]

for floor_index, room in enumerate(office_plan):
    print(f"На этаже {floor_index + 1} работают:")
    for employee in room:
        print(f" - {employee}")

```

5. Когда и где использовать?

Структура	Где пригодится
Список списков	Игровые поля (шахматы), таблицы (Excel-стиль), списки задач по категориям.
Список кортежей	Список контактов (Имя, Телефон), где каждая пара данных — это неизменяемая запись.
Кортеж кортежей	Константы: настройки уровней в игре, географические координаты, которые нельзя менять.

6. На что обратить внимание (Ловушки)

1. Ошибка индексации (IndexError)

Если вы обратитесь к `list[0][5]`, а во вложенном списке всего 2 элемента, программа «упадет». Всегда проверяйте длину вложенных структур.

2. Странная мутация (Списки внутри кортежей)

Это продвинутый момент: если внутри **кортежа** (который нельзя менять) лежит **список** (который можно менять), то этот список **можно** редактировать!

```
my_data = (1, 2, [3, 4])
my_data[2][0] = 99 # Сработает! Изменится список внутри кортежа.
```

3. Читаемость

Если вложенность становится глубже 3 уровней (список в списке в списке...), код становится очень трудно читать. В таких случаях лучше использовать словари или классы.



Практическое задание для закрепления

Задача 1:

Создайте список `cinema`, который представляет собой 3 ряда кресел по 5 мест в каждом (изначально все места — это строка "свободно").

1. Забронируйте (измените на "занято") 2-е место в 1-м ряду.
2. С помощью вложенных циклов выведите схему зала, где "свободно" заменено на 0, а "занято" — на 1.

Задача 2: «Дневник метеоролога» (Средние значения)

Сюжет: Вы помогаете метеорологу анализировать данные. У вас есть данные о температуре воздуха за 3 дня (утром, днем и вечером).

Задание:

1. Создайте вложенный список `temperatures`, где каждый подсписок — это 3 измерения за один день. Например: `[[12, 18, 14], [15, 22, 16], [10, 15, 12]]`.
2. Посчитайте среднюю температуру за **второй день** (индекс 1) и выведите её на экран.
3. Добавьте в список данные за четвертый день: `[13, 20, 15]`.

Ожидаемый результат: Программа должна вывести число (среднее арифметическое) и показать обновленный список.

Задача 3: «Прайс-лист склада» (Кортежи + Списки)

Сюжет: На складе товары хранятся в виде неизменяемых карточек (кортежей), чтобы случайно не перепутать название и артикул. Но сам список товаров может дополняться.

Задание:

1. Создайте список `warehouse`, состоящий из кортежей. Каждый кортеж содержит: (название_товара, цена, количество).
 - Пример: `[("Ноутбук", 50000, 5), ("Мышь", 1500, 20)]`.
2. С помощью цикла `for` пройдите по списку и посчитайте **общую стоимость** всех товаров на складе (цена * количество каждого товара).
3. Выведите результат в формате: **Общая стоимость товаров: X рублей.**

Задача 4: «Школьный журнал» (Поиск и изменение)

Сюжет: Есть список классов, в каждом классе — список имен учеников.

Задание:

1. Создайте вложенный список `school_classes`, где каждый подсписок — это ученики одного класса (например, `5"А"`, `5"Б"`).
2. В класс `5"А"` (индекс 0) добавьте нового ученика `"Артем"`.
3. Из класса `5"Б"` (индекс 1) удалите последнего ученика (используйте метод `.pop()`).
4. Выведите имя **второго** ученика из **первого** класса.

Задача 5: «Карта сокровищ» (Матрицы и координаты)

Сюжет: Вы создаете простую текстовую игру. Игровое поле — это сетка 3x3, заполненная нулями. Где-то спрятан клад (цифра 1).

Задание:

1. Создайте вложенный список `grid`, представляющий матрицу 3x3 из нулей:

```
grid = [  
    [0, 0, 0],  
    [0, 0, 0],  
    [0, 0, 0]  
]
```

2. Попросите пользователя ввести номер строки (0, 1 или 2) и номер столбца (0, 1 или 2).
3. Измените значение в этой ячейке на `1` (как будто вы спрятали там клад).
4. Выведите итоговую матрицу на экран построчно (красиво, в виде квадрата).